Discovery Phase: Benchmark Results & Full-Run Cost/Time Estimate

Prepared: 2026-08-07 · **Scope:** `src/config.py` as committed — `N_SAMPLES=500` x `SAMPLE_SIZE=2000` rows x 7 prompt types, raw-CSV mode.

Bottom line. A 10-call discovery benchmark per model drove two decisions: `SAMPLE_SIZE` was set to **2,000 rows, not 5,000**, and four models were swapped/dropped because they couldn't fit the resulting ~82K-token prompt (`src/config.py` comments). Two local models — `llama-3.2-3b-instruct` (0/10 valid JSON) and `llama-3.1-8b-instruct` (5/10) — have a JSON-reliability problem to resolve before the full run. The full study (500 x 7 = 3,500 calls/model) is estimated at ~**$1,620** across 6 cloud models and **3–6 days serial runtime** per local model; the 5,000-row option would have cost ~**3x** and fit no current local model at all.

1. Discovery benchmark (n=10, `control_prompt`) & full-run (3,500 calls) projection

Model	Parsed OK	Mean latency	Mean tokens/call	Full run est.
`qwen2.5-7b-instruct`	10/10	146.9 s	111,559	142.8 h (~6.0 d)
`gemma-3-12b-it`	10/10	115.5 s	111,031	112.3 h (~4.7 d)
`llama-3.1-8b-instruct`	**5/10**	78.7 s	82,511	76.5 h (~3.2 d)
`llama-3.2-3b-instruct`	**0/10**	72.6 s	82,838	70.6 h (~2.9 d)

No API/server errors on any model — the Llama failures are silent bad-JSON output, not crashes. Full run times are per-model, single-server, serial (parallel hardware -> total ~ slowest model, ~6 d). Tokens/call differ ~35% between model families purely from tokenizer choice, not prompt content (source: `results/benchmark/benchmark_*_n10_*.summary.json`).

2. Why 2,000 rows, not 5,000 — and what it forced

Statistical: at 1,000 rows the bias-label regression is unstable (9% fail) with only ~37% positive labels; at 8,000 rows it saturates at ~94% positive. **2,000 rows gives ~46–56% positive — the best label balance** (`results/ground_truth/calibration_label_rates.csv`). **Token budget:** 2,000 rows ~ 82K tokens/call (fits 128K-context models); 5,000 rows ~ 206K tokens/call (est.), which fits **no** current local model and pushes Gemini 2.5 Pro into its 2x long-context pricing tier. Both arguments pointed the same way.

Forced roster swaps (config.py): `gpt-3.5-turbo`→`gpt-5-nano` (16K ctx too small) · `qwen3-8b`→`qwen2.5-7b-instruct` (32K vs 128K ctx) · `gemma-2-9b-it` dropped (hard 8,192 limit) · `claude-sonnet-4`→`claude-sonnet-4-5` (non-JSON output bug, not token-related).

	2,000 rows (current)	5,000 rows (rejected)
Tokens/call	~82,400	~205,700 (est.)
Fits any current local model	Yes (all 4, 128K ctx)	**No — none fit**
Est. full-run API cost (6 models)	**~$1,620**	**~$4,920 (3.0x)**

3. API cost estimate — full run, 3,500 calls/model

Methodology: prompt re-tokenized with `tiktoken (o200k_base)` → 82,401 tokens/call avg., matching the Llama models' self-report almost exactly (±20% uncertainty for Anthropic/Google, whose tokenizers differ). Output assumed 200 tokens/call. Pricing checked 2026-08-07 — re-verify before budgeting.

Model	$/1M in · out	$/call	**Full run**
`gpt-5-nano`	$0.05 · $0.40	$0.0042	$14.70
`gemini-2.5-flash-lite`	$0.10 · $0.40	$0.0083	$29.12
`gpt-4o-mini`	$0.15 · $0.60	$0.0125	$43.68
`claude-haiku-4-5`	$1.00 · $5.00	$0.0834	$291.90
`gemini-2.5-pro`	$1.25 · $10.00	$0.1050	$367.51
`claude-sonnet-4-5`	$3.00 · $15.00	$0.2502	$875.71
Total			**~$1,622.62**

`claude-sonnet-4-5` (54%) and `gemini-2.5-pro` (23%) drive the bill — input-side rate, not output length, dominates since each call sends ~82K input tokens against ~200 output.

4. Risks / open items

- `llama-3.2-3b-instruct` — 0/10 valid JSON is a hard blocker: fix formatting/add a repair step and re-benchmark, or drop it. `llama-3.1-8b-instruct`'s 5/10 rate would silently lose half its data if run as-is.
- **Cost table is a planning estimate**, not a quote — one tokenizer applied across three providers, single-date pricing. Gemini 2.5 Flash-Lite retires 2026-10-16; confirm its successor if the run won't finish first.